

Code:

```
#include <stdio.h>
#include <stdlib.h>
typedef struct Node { int data; struct Node* next; } Node;
typedef struct mystack { Node* top; } mystack;
mystack* createStack() { mystack* ms = (mystack*)malloc(sizeof(mystack)); ms->top =
NULL; return ms; }
void push(int data, mystack* ms) { Node* newNode = (Node*)malloc(sizeof(Node));
newNode->data = data; newNode->next = ms->top; ms->top = newNode; }
int pop(mystack* ms) { if (ms->top == NULL) return -1; Node* temp = ms->top; int val =
temp->data; ms->top = ms->top->next; free(temp); return val; }
void merge(mystack* ms1, mystack* ms2) { if (ms1->top == NULL) { ms1->top = ms2->
top; return; } Node* curr = ms1->top; while (curr->next != NULL) { curr = curr->next; }
curr->next = ms2->top; }
int main() { int n, m; if (scanf("%d %d", &n, &m) != 2) return 0;
int *arr1 = (int *)malloc(n * sizeof(int));
for (int i = 0; i < n; i++) {
    scanf("%d", &arr1[i]);
}
int *arr2 = (int *)malloc(m * sizeof(int));
for (int i = 0; i < m; i++) {
    scanf("%d", &arr2[i]);
}
mystack* ms1 = createStack();
mystack* ms2 = createStack();
// To reverse the sequence via push, push from left-to-right or
right-to-left
for (int i = n - 1; i >= 0; i--) {
    push(arr1[i], ms1);
}
for (int i = m - 1; i >= 0; i--) {
    push(arr2[i], ms2);
}
merge(ms1, ms2);
Node* curr = ms1->top;
while (curr != NULL) {
    printf("%d", curr->data);
    if (curr->next != NULL) printf(" ");
    curr = curr->next;
}
printf("\n");
free(arr1);
free(arr2);
return 0;
}
```

Input:

```
5 8
9 8 7 6 5
6 7 8 3 11 2 3 4
```

Output:

```
5 6 7 8 9 3 2 11 3 8 7 6 5
```